Transcript

Week 3: Python Fundamentals

Video 1: Python Fundamentals

What if you could quickly analyse numbers to uncover patterns, make informed decisions, and solve real-world problems with ease?

With Python, you can do just that.

As one of the most beginner-friendly programming languages, Python allows you to work with numbers, strings, and logical expressions while performing calculations with ease. Using control flow statements like if-else conditions and loops, you can automate tasks and make your code more dynamic.

Organising data becomes effortless with lists, tuples, and dictionaries, making complex problems more manageable. Debugging skills help you identify and fix errors efficiently, ensuring your programs run smoothly.

Let's get started!

Video 2 - Introduction to Data Structures

Think of organising a messy room or a cluttered bookshelf—everything needs a proper place for easy access. Similarly, programming requires structured ways to store and manage data efficiently. Python offers various data structures that help organize information based on specific needs, making it easier to retrieve and use.

Let's explore these structures and how they apply to real-world scenarios.

Lists, tuples, sets and dictionaries help organise data efficiently in Python.

Think of a list like a grocery list—you can add, remove, and rearrange items as needed.

Tuples, on the other hand, are like ID cards—once created, they remain unchanged, making them ideal for storing fixed information.

Sets work like a box of unique toys, where order doesn't matter, but duplicates aren't allowed.

Lastly, dictionaries function like a real-world dictionary, using key-value pairs for quick and efficient lookups.

Choosing the right data structure makes coding more efficient, so understanding their strengths helps you decide which one to use.

Try them out and see which one works best for your data!

Video 3 - Lists

Need a simple way to store and organise multiple items efficiently?

Lists in Python work like a to-do list—flexible and easy to manage. You can add, remove, and rearrange items as needed. They store ordered sequences and allow duplicates and multiple data types, making them a powerful tool for organising and manipulating data efficiently.

Creating a list in Python is simple and flexible. You can start with an empty list and add items as needed, or initialise it with values from the beginning. Lists can store multiple data types, making them a powerful tool for organising information.

This versatility makes lists a popular choice in Python programming, allowing efficient data handling and manipulation.

Since lists are ordered, you can access each element using its index. Python uses zero-based indexing, meaning the first item is at index 0.

You can also use negative indices to access elements from the end of the list. For example, -1 refers to the last item, -2 to the second last, and so on. This makes it easy to retrieve data from both the beginning and end of a list.

One of the key features of lists is their mutability. You can modify a list without creating a new one. This means you can change, add, or remove items directly within the list.

For example, you can update a specific element, append new values at the end, or remove an item based on its value. This flexibility makes lists ideal for managing dynamic collections of data in Python.

Python offers several built-in methods to manipulate lists efficiently. These methods help with sorting, reversing, counting elements, and more.

For example, you can sort a list alphabetically, count occurrences of a specific item, or find the total number of elements. These operations make it easy to handle lists and perform complex data manipulations with minimal effort.

Slicing is a technique used to create a new list from a specific part of an existing list. It provides a convenient way to extract sublists without modifying the original list. By specifying a start and end index, you can retrieve a portion of the list efficiently.

Iteration allows you to process each element in a list using loops. A for loop enables sequential access to each item in a list, allowing operations to be performed efficiently. List comprehensions offer a more compact way to create new lists from existing ones, making your code cleaner and more readable.

Lists are fundamental tools in Python. Knowing how to create, manipulate, and iterate through them is essential for effective programming.

Now, its your turn. Start practising list operations and make your Python code more efficient!

Video 4 - Tuples

Why store data that can't be changed?

Tuples in Python are like ID cards—once created, their contents cannot be changed. They store data in a fixed order, making them reliable for information that should remain constant. Unlike lists, tuples provide immutability, ensuring data integrity and security while still maintaining sequence.

Creating a tuple is simple. Use parentheses () to enclose elements, just like lists use square brackets []. Tuples can store multiple data types, including numbers, strings, and floats.

If a tuple contains only one element, a comma must be added after the value. This ensures Python recognises it as a tuple rather than a regular variable.

Tuples are immutable and ordered, making them perfect for storing data that must remain unchanged. Since they cannot be modified after creation, they offer better performance than lists in some cases. Their immutability also allows them to be used as dictionary keys if they contain only immutable elements. Tuples ensure data integrity and are ideal for scenarios where stability and efficiency matter.

Like lists, tuples are ordered, meaning you can access their elements using indices. The first element is always at index 0, and you can use negative indices to retrieve elements from the end of the tuple. This makes it easy to access data efficiently in Python.

Tuples are versatile and useful in various scenarios. They are often used for returning multiple values from functions, storing constant data, and serving as dictionary keys due to their immutability.

Since tuples cannot be modified after creation, they provide a secure way to store data that should remain unchanged, making them ideal for efficient data management.

Even though tuples are immutable, you can still perform various operations on them. Slicing allows you to extract a portion of a tuple, concatenation combines tuples, and repetition duplicates elements to form a new tuple.

These operations help manage tuple data efficiently without modifying the original tuple, making them useful for different programming scenarios.

Choose tuples when you need data to remain unchanged throughout the program. Use lists when you need flexibility to modify the collection. Understanding these differences will help you select the right data structure for your needs.

For data that shouldn't change, tuples are a powerful tool in Python. Learning to use them effectively will improve your code's reliability and efficiency. Put them to the test in various coding challenges to truly understand their benefits.

Now it's time to put your knowledge to the test! When should you use a tuple instead of a list? Try implementing both in different scenarios and see the difference for yourself.

Video 5 – Sets

Ever needed to store a collection of unique items without worrying about duplicates?

Sets in Python are like a bag of unique items where order doesn't matter, and duplicates are not allowed. Similar to mathematical sets, each element appears only once.

Because sets are unordered, their elements don't follow a specific sequence. They are also mutable, meaning you can add or remove elements as needed. This makes sets ideal for storing data where uniqueness is required.

Creating a set in Python is simple. You can initialise a set using curly braces or the set function. However, be careful—using only the curly braces creates an empty dictionary, not a set.

Sets can store multiple data types, including numbers, strings, and floats. Their flexibility makes them useful for managing unique collections of data efficiently.

Sets have distinct properties that differentiate them from other data structures like lists and tuples. They are unordered, meaning elements do not follow a specific sequence and cannot be indexed or sliced. Sets automatically remove duplicate values, ensuring all elements are unique. They are also mutable, allowing the addition or removal of elements, but they cannot store mutable types like lists.

Python sets support various operations similar to mathematical set operations. They are mutable, allowing you to add and remove elements. The add method inserts a new element, while remove and discard delete elements. The key difference is that the remove method raises an error if the element is not found, whereas discard does not. To clear all elements in a set, use the clear method.

Python sets support mathematical operations similar to those in mathematics.

Union combines elements from both sets, while intersection finds common elements.

Difference identifies elements present in one set but not the other, and symmetric difference returns elements that are unique to each set.

These operations make data analysis and manipulation easier.

Sets are useful when working with collections where uniqueness and membership are important. They automatically remove duplicate values, making them ideal for managing unique items. Sets also provide an efficient way to check membership and perform mathematical operations on datasets.

While sets are powerful, they have certain limitations. Since they are unordered, you cannot access elements by index. Additionally, sets cannot store mutable elements like lists or other sets. Another limitation is that operations like addition and removal do not raise errors if an element is not found, which can lead to logical issues. These factors should be considered when deciding if a set is the right data structure for your needs.

Sets are a valuable data structure for managing unique elements, but they come with certain limitations. Understanding these constraints will help you decide when to use them effectively in your programming tasks.

How would you decide whether a set is the best choice for a particular problem? Think about it!

Video 6 – Dictionaries

Have you ever needed to store data in a way that makes retrieval fast and efficient? What if you could organise information using key-value pairs for quick lookups?

Think of a dictionary in the real world, where each word is paired with its definition. In Python, dictionaries work the same way, but instead of words and definitions, they store key-value pairs. Each key is unique and immutable, such as a string, number, or tuple, while values can be of any data type. Dictionaries allow for fast and efficient data retrieval, making them a powerful tool for organising and managing data.

Creating a dictionary in Python is simple. You can define one using curly braces with key-value pairs or use the dict function. Since dictionaries are mutable, you can add or remove elements after creation, making them a flexible data structure for storing and managing information.

You can access or modify elements in a dictionary using keys. Add new key-value pairs by assigning a value to a new key, update existing values by reassigning them, or remove elements using methods like del or pop. This flexibility makes dictionaries a powerful tool for managing structured data.

Dictionaries have several built-in methods that simplify data manipulation. The get method allows you to safely retrieve values by providing a default if the key is missing. The keys, values, and items methods help you access all keys, values, or key-value pairs within the dictionary, making data retrieval efficient and structured.

Dictionaries are versatile and useful for various programming tasks. They allow you to store structured data, count the frequency of items, and group data based on common keys. Their flexibility makes them ideal for managing and analysing information efficiently.

Dictionaries can contain other dictionaries, creating a nested structure. This allows you to represent complex data relationships, such as hierarchical structures in databases or organisational charts. Nesting makes it easier to manage and access grouped data efficiently.

While dictionaries are powerful, they come with certain limitations. Keys must be unique and immutable, and dictionaries are unordered collections, meaning they do not support indexing like lists. Additionally, they can consume more memory than lists or tuples. These factors should be considered when deciding whether a dictionary is the right data structure for your needs.

Dictionaries are a great choice for many tasks, but knowing their constraints helps you use them effectively. When would you choose a dictionary over other data structures?

Video 7 – Functions

Have you ever found yourself writing the same block of code multiple times? What if there was a way to simplify your program while making it more efficient and readable?

Functions are like kitchen appliances—each is designed to perform a specific task. In programming, a function is a reusable block of code that executes a specific action, preventing repetition and keeping your code organised. Functions help in code modularity, simplify complex problems, and improve readability.

To define a function in Python, use the def keyword followed by the function name and parentheses. The function body is indented and contains the code that executes when the function is called. This structure helps in organising code and making it reusable.

Parameters act as placeholders for data passed into a function, while arguments are the actual values provided during a function call. Defining parameters allows flexibility in function execution. Python also supports default parameters, keyword arguments, and arbitrary arguments for greater control.

Functions can return values using the return statement, allowing you to send results back to the caller. This is useful when you need to store or use the function's output elsewhere in your program. If no return statement is specified, the function returns None by default.

In Python, scope determines where a variable is visible in your code. Variables declared inside a function are local and only available within that function. Variables declared outside any function are global and can be accessed anywhere. You can also use the global keyword inside a function to modify a global variable.

Lambda functions are small, anonymous functions useful for short, single-use tasks where defining a full function is unnecessary. They are defined using the lambda keyword, followed by parameters and an expression. Lambda functions are commonly used in situations requiring quick, one-line operations.

To write effective funct_ions, keep them focused on a single task, use descriptive names, and avoid relying on global variables. This makes your code easier to read, maintain, and debug.

Functions are essential for writing clean, reusable, and modular code in Python. Mastering functions will help you improve your programming skills and solve problems more efficiently.

How can you use functions to make your code more efficient?

Video 8 – Loops

Why write the same code over and over when loops can do it for you effortlessly?

Loops are a key concept in programming that allow a block of code to repeat multiple times. Think of them like a washing machine cycle—repeating until your clothes are clean. Loops help automate repetitive tasks, reduce redundancy, and improve code efficiency.

A for loop is used to iterate over a sequence, such as a list, tuple, or string, and execute a block of code for each element. It is ideal when you know in advance how many times the loop should run.

The range function is commonly used in for loops to generate a sequence of numbers. It is useful when you need to iterate a specific number of times.

A while loop continues executing as long as a specified condition is true. It is useful when the number of iterations isn't known in advance, allowing execution until a condition changes.

You can control the flow of loops using break and continue. The break statement exits a loop immediately, while continue skips to the next iteration without executing the remaining code in the loop.

Nested loops occur when a loop is placed inside another loop. They are useful for iterating over multi-dimensional data structures, such as lists within lists. Each time the outer loop runs, the inner loop completes its cycle.

To write efficient and readable loops, always ensure that loop conditions will eventually become false to prevent infinite loops. Use loops to simplify repetitive tasks, reduce redundancy, and enhance code clarity.

Loops are a powerful feature in Python, helping automate processes, manage large datasets, and improve program efficiency. Mastering both for and while loops will allow you to write cleaner, more structured code that adapts to different scenarios.

Now that you've learned how to use loops effectively, how will you apply them in your next coding project? Try writing a loop today and see how much time it saves you!

Video 9 – Student Record System with Typing in Python – Demo

Hello and welcome everyone. Today, we are going to do a complete hands-on demo to consolidate all the Python concepts you have studied till now, the data structures, the functions, loops and I am also going to introduce you something called as typing. We are going to build a student record management system and we are going to use all the fundamental data structures.

So, let us start with building this student record management system. To start with, I am going to actually introduce you a new concept typing and I am going to show you that how this is helpful in the readability of the programme. So, because we are going to use these different data types, I am going to import them from typing.

Now, the moment I do so, they are actually type definitions and let me show you how that is helpful. I am going to create a student ID and I am going to mention the type of that. Similarly, I am going to create a student record and I know that it is going to be a tuple of a string and a dictionary of a string followed by its value as an integer.

Why do I say so? Because I am planning to include the record as the name of the student the subject and the marks associated with that. That is going to be my schema of a record of a student and this schema is very appropriately defined over here. It is going to be a tuple of the name of the student, the corresponding subject and the marks stored.

Further, I am going to define a database which is going to be a dictionary. You know that database is more or less like a dictionary where I have a student ID and the

corresponding record of that student. So, you see that to define the schema, we leverage this typing or type definitions in Python.

It is very helpful and in the very initial step, you have defined the schema, you know how your variables would look like, you know how the record would look like. Anyone who is about to look into your code would also have the same concept and all of you would be on the same page. Let us say I define a global database; I will say students DB which is the database of all the students.

It is going to be a database type and I am going to have an empty value for now. Now, I go by starting by defining functions. You know that to define a function, I use the word def as a keyword.

I will use the name of the function. I will say let us say I want to add student and it is going to take a database object and it is not going to return me anything. So, it is going to consume a database where I am going to basically add the student into the database and I do not need return anything.

So, I am just mentioning none. This is what typing is all about. It more or less defines your function in terms of what is it going to consume and what is it going to produce making it easy for users to read it and to manage it.

Now, you know that when I am about to add a student, the first thing which I would need a student ID and you know that a student ID is an integer type. So, I am going to ask the user to input the student ID. Similarly, I can do that to the name and you know that name is going to be a string type input.

Once I have the student ID and the name, I would also want to do a quick check. If the student ID is already in my database, then I will simply print student ID already exists and I just return because I am not returning anything. So, I do not return a value, I am just going to return out and almost the same thing I would do.

Well, we do not need to do that with name because ID is something which is a primary key to my database. The next thing which I want to do is the number of subjects the student has to enrol in. So, let us say subject count.

Now, you know that subject count is an integer. So, I am going to mention the type and I am going to ask the user to input the number of subjects. Once I have the subject count, I would start with getting all the details of the subject.

Now, I am going to use another typing, I am going to say subjects which is going to be a dictionary type with a key as a string and a value as an integer and it should be an empty dictionary. I am going to loop over all the subjects and I am going to ask the user to tell me the subjects, to tell me the marks for the subject and I am going to add that to my subjects dictionary. So, now I know that for a given subject what are the marks scored by the student and I am going to add that to my database corresponding to that student ID.

The name of the student and the subjects which would contain the different subjects within it and their corresponding marks. So, that is how simple and I can finally say that print student added successfully. That is my add student function defined.

Similarly, I would do that to view a student. Let us say after adding it, I would want to view the students. I am going to consume the database back again and I do not need to produce anything in the output; I just need to view the students.

Now, I will just put a valid condition that if there are no student records, I should check that if there is no database which exists. I will simply say that are no students records found, but if that is not the case and I have students. So, I am going to move through all

the student ID and in fact, I would say use the tuple here the name comma marks across all the database items because my database is a dictionary.

So, I will loop over its items and I would mention here print statement, which is going to give me, let us say I start with the ID of the student. So, I would say student ID and its corresponding name. In fact, let me have that over here itself and then I am going to list the marks of all the subjects for that student.

So, the subject and their corresponding marks. So, I would say subject let me have that in the parentheses and the marks something like this. Now, that is how I would print all the entries.

So, that is how I would enter the student ID or in fact, we are not entering a student ID, we can actually view all the students in my database. Now, let us say I want to do it for a specific student, I want to search for a specific student. So, I will first ask the user to provide me the student ID.

Now, if the student ID is not found in the database, I would basically simply say that student not found. Otherwise, I would go by fetching the details of that student. So, I would go to my database, search for that student ID, I would get the name and the marks and I can simply say that for this student ID, this is the name and the corresponding subject and marks for that student.

So, that is how I can actually perform the search operation also. So, I have basically defined these three basic functions and you see that how we have used functions, how we have used the if conditions and the for loops. Let us now define the main loop, I am going to say define run app which is going to run the app.

Now, here I am going to start with a condition, let us say I want to add a student, view a student, search a student. Now, I would ask the user to enter a certain choice. Now, if the choice is 1, I would want to add the student, if the choice is 2, view the student, if the choice is 3, search a student and if the choice is 4, I would break the loop.

Sorry. Now, I run this app and let us see if it all works fine. I would want to start by adding a student.

So, I would say 1, I would say maybe something like 100, I would say the name is Ravi, the number of students subjects are 2, I would say Python and I would say let us say out of 50 students scored 49 in Python, I would say data science and let us say the student scores 40. So, the student got added successfully. Now, if I want to view the student, I would simply say 2 just to see if my database has updated that or not and I do see the ID is 100, the name is Ravi, subject Python's number is 49 and data science we have a score of 40.

I would want to add more students over here, I would say 101 now. In fact, I would want to add a student 101 now, let us say I would say Ramesh, I would say again 2 subjects, let us say Python 45, I would say machine learning 40. Now, again I would want to see the students, yes, we have Ravi and we have Ramesh and the corresponding marks.

And I want to check the third choice also to search a student and I would say 101 and it does list me, Ramesh. So, looks like we have all our features of adding, viewing the added students and searching them correctly working. There are slight formatting issues, but I would want you to fix them.

So, basically you see that how we have managed to create a small application to maintain the student records. You could also go by deleting a student, you can also go by more functionalities like calculating the grades, you can also go by finding out the topper student, finding out the average per subject. These are some more inputs which I would want you to try and see if you are able to perform those.

We are going to provide you a solution with all these extra features, but I would request you to try it on your own and then use the solution as a reference. If you are stuck, you can use that as a reference to further move ahead in your code. That is all guys.

Good luck. Thank you.